

ActInf ModelStream 015.0: Andrew Pashea "Active Agents: Agent-Based Modeling in the Social Sciences"

Source: [YouTube](#) Playlist: ModelStreams Duration: 1:27:08 | Uploaded: Unknown Transcript method: auto_caption

hello and welcome this is active model stream 15.0 with Andrew Pasa from ABM to active inference inference-based action selection so Andrew thank you for joining looking forward to this presentation and overview so go for it awesome thank you so much Daniel uh yeah again I'm Andrew a and I uh I am a full-time data analyst SL scientist engineer uh many roles to fill when working in the nonprofit world and uh so a couple months ago I gave a uh a workshop like a full uh talk on constructing active inference agents using the pmdp library as we'll be using here today um and uh during that talk I described how we might be able to employ active inference agents uh whenever putting together agent-based models uh in the context of computational social science and so you'll see some links in the description referring back to that tutorial that I gave including my GitHub which has all uh tutorial materials uh on that page as well as some in browser Google collab scripts that run python code and so you can actually follow along um a lot of these uh learning resources including this talk I'm giving are sort of part of a personal initiative I've taken to try and uh create and develop and then share open- Source learning resources for applying active inference um I see so much potential as someone coming from more of a social sciences economics oriented background uh you know for for applying active inference agents in areas where maybe they haven't been employed too much before um and as active inference is beginning to be increasingly applied to uh social science research such as uh you know attempting to study and and simulate uh something like human behavior and decision making uh I thought that that conference was a was a really ripe opportunity to try and uh make those kinds of connections for people and and just kind of get those kinds of dialogues moving um and so for me as someone who came to active influence with that kind of background but also strong interest in cognition um I just really wanted to uh kind of do some from scratch sort of tutorials on how to construct agents um you know as someone who's been interning with active inference Institute for nearly a year now um and I'll

also be giving a talk at her upcoming posium which itself will be a kind of extension of that tutorial I gave a couple months ago as well as the talk I'll be giving now um yeah I would just like to uh really make it plain for people sort of the general mechanics that go into constructing active inference agent uh I would also refer people back to the textbook that was published a couple of years ago by uh pulo par and Carl friston um it's freely accessible you'll find links to it in my uh slides as well as in the collab scripts if you follow the GitHub link um a lot of material that I'm using to construct these tutorials is actually coming from that textbook so anyone involved with the Institute uh reading group uh where we cover the textbook will uh kind of feel hopefully a little at home whenever they follow these tutorials um so we'll start off now um we'll be constructing uh the longer span of the tutorial is one uh make the connection between active inference and agent-based modeling agent based modeling is a very common Paradigm in computational social science uh that said and this is not doing it justice but I'm saying this just to keep it brief uh in some ways it's very traditional in the uh kinds of assumptions that it makes more often than not about human behavior uh often times people rely upon a kind of homoeconomicus or or self-centered uh utility maximizer kind of a description of what defines human decisionmaking and um I think that that loses a lot of the sort of richness that we find in cognitive science neuroscience and and Behavioral Studies uh of how people might actually make decisions and active inference supplies us with you know um a rather you know simple assumption it's a very complex assumption that being free energy minimization whenever you break it down but ultimately that's a core key assumption that we're making and we don't have to make a ton of additional assumptions beyond that whenever considering how to simulate uh uh human behavior Beyond how do we construct our model right or how do we construct our agent and so um that's that's sort of the connection that I'm making here between agent based modeling and activ inference then after that I describe how what I did with that tutorial was I recreated uh this kind of paradigmatic uh simulation published back in 2007 uh by uh by doctors laser and fredman uh you'll see links to that paper as well at the GitHub site uh through the link and uh what they do is that they set up what I would deem to be a so-called traditional uh social science agent-based model where um agents are basically pre-programmed to do particular things um they don't exhibit autonomous decision making which is sort of the promise of a lot of generative AI today uh and I think that active inference especially emphasizes that sort of autonomy in the sense that these are agents who are you know minimizing their own free energy and then from there you know predicting what should be um the next action that they

should take or or set of actions meaning a policy that they should take and uh and so they engage in all these kind of cognitive uh functions or or mechanisms that we we find humans to have related to to planning and um uh the idea of having habits that you build up over time and how those sort of compete with more deliberate decision making and um you know the kind of uh executive functioning in the prefrontal cortex of the brain and and finding those kinds of connections so um without further ado what we'll do now is simply create one agent and um we'll I'll go through the steps of how to do that and uh this is the agent that features in the uh laser Freedman model Recreation that I do where I use active inference agents uh as opposed to the kind of pre-programmed agents that that they use um so um we'll start with saying again this is pmdp the link is there we import some libraries um It's relatively not too many and some of them are just for like the plots um the main thing is you can pip install pmdp uh there'll be mentions later but there have been many updates to that Library over the past couple of years uh and a little bit more context on PDP as far as I understand it from a couple conversations with the developers is that this package is uh directly inspired by and in certain ways trying to emulate the mechanics of uh the SPM package uh developed by by friston at all uh in mat lab for for constructing active inference agents and environments and processes so um it's it's it's sort of the Exemplar python library for doing active inference as far as I'm aware um and that said there's so much complexity whenever uh these libraries are constructed and a lot of you know dependencies between different like subl libraries and so on so it it becomes really useful to find more focused sort of snapshots of how to construct say an agent and understanding how those mechanics work without necessarily having to scavenge through the code that said uh there are some examples and example tutorials given at the PDP website it that I've linked there on the screen uh so please uh feel free to peruse those um and so moving on from there uh for anyone familiar with the textbook uh it's very useful to refer to Chapters six and seven uh chapter six refers to uh kind of providing a general recipe for constructing uh a agents and environments and running simulations um and uh then from there chapter 7 kind of expands upon those ideas by specifically creating partially observable markup decisionmaking proces processes or uh pdps after which pmdp kind of cleverly takes its name and um those are the kind of discrete time discrete State space uh models that we'll be working with that is they they uh play out uh in action perception Loops uh one time step at a time as opposed to it being continuous time so you could say you know at time step one or t equals 1 uh and then at time step two this happens times step three this happens and then discrete State spaces meaning that uh we're not looking at continuous variables from

continuous distributions per se we're looking at uh probability Mass functions and you know just more concrete like you know there's a hidden State uh Factor called color and it has two levels red and blue you know these kind kind of definitive discreet uh uh factors and levels so uh to construct an active inference agent uh probably the this is the general outline all smashed into one slide uh so the first three steps defining States defining observations defining controls action policies um with the nuances for all three of those additional terms so States what are our agents uncertain about uh what kind of you know sets of beliefs do they hold that they end up inferring and updating potentially learning over time uh and then defining observations what do our agents observe from their environment right um it's not enough to like claim that you have an active inference agent and throw it into some you know ethereal space where it has no sensory receptors no sensorium you know you need to be able to Define like what are uh senses so to speak that the agent has available uh the technical phrasing being observation modalities so for example uh an agent might have um you know a visual uh uh observation modality where it can take in discrete levels uh rain or no rain meaning at any given moment it's observing if there's rain or no rain uh through its visual observation modality or visual sensory receptor being able to clearly Define those kinds of connections almost as if we could view them on a graph which we will do momentarily um and then defining control action policies for all intents and purposes going forward I'm going to be relying upon the word actions um that's because actions are simply in this case discrete actions an agent can take at a given time step um there's nothing more to it than that uh you can walk or you can run or you can sit uh those would be three actions that an agent could do at any given time step right they're not doing multiple of those in the same time step and they're not uh doing them in any kind of deterministic way so to speak um so with controls that's a term that comes up very frequently in the literature and it also relates to other ways of doing say uh reinforcement learning related to engineering problems or otherwise but the the idea of a control is that it's just an action with kind of an additional Nuance to it that uh it relates to the idea of this is an action that can actually change or control something that something being a hidden state or hidden State Factor um so it's you know whenever kind of putting together the matrices involved in constructing PDP agents uh there are times where you'll need let's say a and we'll I'll clarify what some of these words mean for for anyone who's who's still new to to uh modeling here um but uh so with controls like there might be it might be the case that you need some kind of subarray like a b Matrix or submatrix that um you know doesn't really do anything like it might be a hidden state for a rat in a teamat uh who you know they they know

that they can control which location of the teammates they go to they could go to the left arm they could go to the right arm but they don't necessarily think that they can change the location of the reward it's always either on the left or it's on the right and you don't know until you see it right so it's like the control there would be choosing the left or right arm and the left or right arm is the correct option being like the uh or rather the the left or right arm in state where the rat infers if it's gone left or gone right like that is something you can control moving on with policies policies are sequences of actions in this case you could have um you know in the teams you have a rat who can choose to go left and then stay there it can choose to go right and stay there it could choose to go to the queue and then go left uh or otherwise but the idea is it's just a sequence of actions uh and so uh agents can have their own sort of hypotheses so to speak about individual policies and that's very interesting to think about whenever we're considering active inference agents who you need to engage in kind of longer term planning Horizons and and who might be used for you know imagine uh for people who are into engineering or or otherwise might be looking at a like a like scheduling problems or you know inferring situations where you know that you're going to need to do a series of actions in a sequence that maybe need to have some kind of coherency with one another such that it's useful to refer to that specific specific set of actions as a policy um so Define States Define observations Define your actions after that we relate those three components of a model uh via a series of uh categorical probability distributions which are in the a b c d and e uh matrices um again all these feature in the the textbook and then uh th those will comprise the beliefs of the agent then we Define our environment which here we're not going to uh for through the sake of today for the shorter uh talk we're we're not going to get too far into defining an environment uh I will say that common ways of defining environments especially whenever you're just beginning the modeling process is that oftentimes people will actually construct another PDP which itself represents the environment because in that way can take in its own observations those being the actions of the agent and then it can output its own actions those actions being the observations for the agent that's forming kind of squaring the circle on our action perception Loop um but uh for the sake of today we'll just be looking at a fully deterministic environment where I manually have it take certain inputs send out certain outputs it's still an environment it's just not a very complex one uh and then after that we just Define the active inference Loop run it as a simulation runs the full loop agent receives observations in first States it learns first policies it commits in action then after that uh good modeling practice if you're actually trying to to do something with what you're modeling uh

and and trying to model some real phenomena is that you would look at the results of your uh simulation you would look at what makes sense to you what doesn't Maybe maybe there's some parameters that uh don't quite make sense that you've set up for the agent maybe it's missing an observation modality um or or otherwise or uh if you're you know using these simulations in a in a kind of more productive way to to derive insights then the idea is that you could tune things and you could modify the environment as if you're you know um uh uh introducing some kind of intervention and then you could see how things play out differently right in the case of people who are maybe concerned with with policy design or or kind of analyzing uh agent Behavior under uh various kinds of conditions uh the agent we'll be constructing today is uh an agent whose task is to find a good solution to a complex task of problem uh the idea is that uh the Fuller idea is that they're trying to navigate what's called an ink landscape which is you know you could you could visualize this as a broad set of solutions and each of them has its own Fitness value so we're looking now at a fitness landscape at any given point in time in the Fuller simulation which I'll cover in maybe future talk is that at any given time an agent has a solution currently um it can for the sake of this talk I'll say it as simply as this the agent can either at each discrete time step during the action perception Loop they can either explore meaning they try to find a new Solution that's better than the one they currently have or they can exploit their neighbor which means they steal their neighbor answer um this is for more context this is useful for understanding like if you wanted to uh put agents into a network where they interact with one another which is precisely what my multi-agent simulation from the tutorial does again uh review the GitHub uh all the code is there for doing so uh the rest of the slides in this particular slide deck also uh get into that but we're only focusing on these um so the the agent can explore or exploit and a quick note as well that let's be cautious not to confuse the use the words explore and exploit here with how they appear in a lot of the literature uh on the explor versus exploitation problem um here we're just going to say those are the names of the agent actions they could be replaced with study or cheat right um but I'm just I recreated uh the laser Freeman model they use explore exploit it makes sense in a particular way and so we're going to stay with that um after that the agent will infer an intention hidden State uh that is they'll they'll receive observations uh from their environment they they they chose to explore or exploit they receive observations as a response from their environment they'll infer a hidden State and then they'll learn meaning they'll update the the the parameter values of their a matrix or likelihood Matrix um move to this this is much more representative of the process so we can start at the top the agent explore or exploits

environment takes in those actions and then based on uh based upon whatever rules you set for it to have or you could again you could construct some kind of you know generative model um that you know will kind of do it on its own uh follow you know you could add stochasticity or some kind of you know error term or something so that it's you know it's it's not entirely deterministic however you like um but it will generate observations at any time step the agent after committing an action it being sent the environment the environment processing that action the environment will send back two observations to the agent um these upper to Improvement or no improvement showing that oh the agent explored for example and then saw an improvement in its solution to the problem and then or no improvement you know and then uh the other observation is self or neighbor which is a one to one uh uh mapping that is if you explore it's going to return self if you exploit it's going to return neighbor the idea is this agent has two observation modalities one of the first one relates to the quality of their solution uh and and outcome of their action uh and and then the second observation modality simply is a kind of self-reflexive uh observation which is who did I attend to after that the agent will take those observations to infer hidden state it only has one hidden State Factor uh that contains two levels self and neighbor uh there will be again a one to one mapping between self and neighbor if you see the outcome you attended to yourself then you're going to infer that you attended to yourself vice versa for neighbor um and then in addition to that inference learning occurs uh which uh modifies the uh categorical probability distribution describing uh the the probability of those outcomes given State beliefs and the agent from there infer actions or policies you should take and carry them out and we proceed so this and there will be times that I might Zoom through particular slides um but in this case this is one that's good to reflect on um something that I think a lot of people Miss uh or or maybe struggle with uh having been involved with the Institute the reading group for some time and having my own trials and errors and in learning how these things work uh is that there's a lot of nestedness we're working with a lot of uh matrices uh anyone who not only works with PDP uh but also math lab and SPM is is very familiar that it's it's crucial to be able to understand how to construct matrices how to construct matrices that represent categorical probability distributions I.E for the agent their beliefs um and uh and so it's it's really important to wrap your head around that and so I just really wanted to spell things out and there's an additional collab script that I'm currently writing that already has now a bunch of markdown cells so that you can not run the code uh but you can see like full descriptions on how these things work um all I'm basically adding more things to to that script that that weren't available uh to me or or I just didn't have

time to include uh one of our devices initial tutorial um but now that we've uh defined our hidden States observations and controls um we'll link them together through the a b c d and e matrices and so hidden States uh a clarification on terminology uh agents will have hidden State factors they're they're called hidden State factors they'll have uh you know say and hidden State factors and uh and and so our agent has only one it's the attention hidden State Factor containing two uh discrete levels uh self and neighbor you see the same exact kind of nestedness with OB observations and observation or sometimes referred to as outcome modalities so you'll have M observation modalities our agent has two of them one of them is for outcomes uh which uh the discrete levels of which are Improvement and no improvement as I showed before um this code actually includes a third one called unobserved uh it never comes up in any of the simulations I run I thought it would be interesting though to include it if only because it shows that you know you can include uh additional dimensions in your model even if you don't know if they're going to come into play at any point the simulation uh it can potentially change things depending on how you set those initial values in in the uh belief distributions but uh you know for the time being I just wanted to show that uh it's possible to do that and finally uh controls and control factors uh so actions uh you know again as I earlier an action is just a discrete thing an agent can do you could have similarly with nestedness you could have a whole control factor of different actions an agent could take with a total of f control factors our agent will only have one and it'll contain two discrete levels or discrete actions explore and exploit as far as defining hidden State factors hidden States observation modalities observations and then controls or actions um this is it this is all the code youi um that's part of the and I'm following a certain kind of GOM clature that PDP uh the developers use in their tutorial website it's a it's a very clever way of keeping track of things um none of these string values like self it's uh Improvement none of them matter so to speak um they're just a good point of reference by uh including them in a list so this first list attention state names represents the hidden State Factor uh for attention and it just it the list contains two discrete levels self or neighbor I included one in this simulation to note you have one neighbor so neighbor one uh and then from there you just store uh the number of states uh for each instate Factor uh this is appears a little redundant because we only have one but I'm just you know this if you were to add like say multiple hidden State factors this would be more interesting um and then tracking the number of hi State factors that you have um for anyone who unfamiliar with Python Programming the len method allows you to just see what is the length of the object that you're inputting so we only have uh one hidden State Factor

which leads us to uh one the next small code snippet very similarly now we Define each observation modality the outcome observation modality has three discrete levels Improvement no improvement as I mentioned earlier unobserved the attention observation modality contains two discrete observations cell for neighbor one so recall that in a given time step an agent is going to receive one of each of these so at uh you know time step four the agent could receive Improvement and name one or could receive no improvement and self or unobserved in s the point is that it will receive one of each right um again we track those using uh the length or Len uh function and then once again actions uh we put them we only have two actions so I just referred to it as the action uh control factor and we included our two actions you can explore or you can exploit neighbor one and we track that the number of controls available to the agent um again that that's it and and if you can determine you know this kind of design for your agent early on uh and then begin programming I mean as as the authors of the textbook acknowledge uh it's quite an iterative process whenever it comes to to computational modeling of just about anything that includes active inference agents and and running uh you know simulations and and so on so um this is a very crucial First Step because it'll inform a lot of what you do next as well as you know what you might end up changing during that process of of iteratively you know changing your your model of your model so the a matrix is the first step of linking these things the a matrix will link together the observation modalities and hidden State factors uh that we defined earlier we need one sub array for each outcome modality so I want to just draw attention to this upper right equation um again Matrix operations are perhaps the the main hurdle in learning how to construct imdp um agents or any PDP agents including matlab um so a useful rule of thumb is just remembering that whenever you construct uh a matrix to represent say U you know the a matrix the likelihood Matrix uh you know that the probability of whatever outcome you're looking at whatever observation modality you're looking at we'll start with the outcome one which again uh includes Improvement no improvement none observed the probability uh of any of those observations conditioned on every single one of your hid State factors which nicely for us is as simple as uh the the way it looks to the far left which probability observation modality outcome uh conditioned on the hidden State factor for attention right so we're really just uh working with these these two Dimensions um it it would become threedimensional matrices if say we had two hidden State factors this would get expanded um so uh the typical way of doing this in PDP uh PDP contains a subl library utils uh containing just different utility functions and the the sort of framework for constructing these kinds of uh beliefs in p M DP is by using what are

called object arrays which are already available in the numpy library uh it's effectively a numpy array that allows for uh data types aside from float values so it's kind of a more flexible version of a numpy array um and that allows you to do things like say create um an a matrix that's your representative of your overall likelihood Matrix for everything all observation modalities and state factors and then you can construct within it individual subarrays one for each observation modality and that those subarrays can be of different uh dimensionalities from one another and so it's it's just again wrapping your head around kind of the nestedness here um and a lot of these things are better represented in the the code I've written so might touch on that if there's time um uh of course this is code I wrote too with the the collab script which uh contains code that you can run and it has different ways of expressing these different matrices but you can see the point is that we're now that we've defined our hidden States our observations our actions we're going to start linking them and to represent our agent beliefs so uh for this situation I print the outputs all code that you've seen so far with the gray shading that is uh you know at the beginning we imported libraries and then two slides ago we um we looked at defining H State factors observation modalities actions now this is our a matrix um I'm not getting into the maths on this a matrix because it's easier to just look at the outcomes um this is how we've construct like all this code that I've shown so far can just be run like in the order that I've shown it thus far you'll see that in the collab script um here the first subarray again which is for the outcome observation modality we'll notice for those who are familiar with probability Theory um probability distribution sum to one uh that that's help what's helpful is that the utils uh subl library has is normalized uh function where you can put any object array into it it'll tell you if it's normalized or not so helpful rule of thumb to ensure that you've done everything right before you proceed further um and so what this does is that it lets us see that if we sum these values um each outcome uh per pin state level uh we can see that it's entirely uniform it's it's totally uncertain it's not useful to the to the agent there's a third probability that they'll see an improvement if they attend to themself third probability if of there being no improvement if they attended themselves there's a third probability of seeing an unobserved outcome if they attended them um it's very common practice if you're attempting to model a more realistic um you know phenomena or or or organism or human and you have no idea what they would do uh it's CA practice initialize them with uniform priors here I'm just doing it for the sake of Simplicity and a in a tutorial right um and so uh meanwhile with the next subarray as I said earlier uh there's one toone mappings between the uh attention observation modality and the attention hidden State Factor so it it's as if the agent has

completely certain beliefs like the polar obosa uh about um about who it's attending to right so if it sees that it's attending to someone um attending to itself then it will infer its intend attending to itself and vice versa for neighbor and that's it uh that's the a matrix and then we have the B Matrix which I would say is equally complex and then Things become immensely simpler from the B Matrix again equation top right um uh we need one subarray uh B Matrix for each hidden State Factor this is very simple we only have one hidden State factor it's the attention hidden State Factor um and so our agent wants to be able to track um if I'm currently attending to let's say myself and I choose to explore what will the hidden state be at the next time step thus s to plus one conditioned on sent the hidden State present uh combined with π which is representative of the action or policy undertaken in the present um this is also going to be incredibly simple it's really going to be one to one so at any given point in time um the agent will believe that regardless of what state it in uh currently it will transition to self if it chooses explore probability one if it was in if the current state is neighbor it'll transition to self with probability one if it chooses explore and similarly it always believes that it'll transition to the neighbor in state if um regardless of whatever in state it currently believes it is in so uh a a comment that that might come up for anyone who is familiar with these things this is not terribly uh different from just a standard Markov decision-making process right it's kind of like well if the agent has all these fully certain beliefs why create a partially observable environment um there there there's a certain reason for that and then there's another just more direct reason for that the direct one is that this is how pmdp Works um and then also I the the other reason is I think that partially observable environments are more representative of our reality I think it aligns with the idea that we do often have noisy sensory receptors uh you know whether you have 20/20 Vision or not uh it doesn't mean that you can uh you know see every color that's every uh ever existed and it doesn't mean that you know you can see perfectly anything uh and so the idea of just this noise being there and the idea that you have to infer you have to rely upon observations to infer beliefs and having those kinds of Loops you know it all um kind of makes sense and then there's another aspect uh that will come up soon whenever we discuss learning for the C Matrix which is uh representative of preferences or priors over observations this is basically denoting uh it's as if we're saying uh what are the outcomes the agent actually wants to see like or observe what are the what are the observations it wants to to to see happen uh as oppos to those that it doesn't right and so this is immensely uh easier to construct uh versus the the a matrix or the B Matrix um in that we're really just effectively creating these vectors um so we need one subarray for each outcome

modality or observation modality for the first one uh the outcome modality the way that we've defined this is take a list uh each one of these values is indexed to the observation in the outcome uh observation modality as we saw before so recall in order the first one is Improvement the second one is no improvement and the third one is unobserved these line up exactly uh in the same way and uh reiterating Again The Matrix operations uh and and being able to grasp like how indexing goes here uh as long as everything lines up uh it'll be solvable right the the free energy calculation will be able to be carried out as long as everything kind of is in its place right so so it's uh do your best to never confuse uh you know which position each of these represents so the reason why these values don't sum to one uh common practice uh that I've seen a lot in previous mat lab scripts and so on is U uh people will actually use these like whole integers and even negative values because it's just kind of a more human friendly human interpretable way of of experimenting with an agent's preferences um whenever you run this through the soft Max function uh which which is also applied here this basically strongly amplifies um the the final probability distribution that that arises and also turns it into a probability distribution that does SU to one um and then uh before we move on to see what that looks like for the second observation modality uh the attention observation modality we're going to make that entirely uniform as well what each of these really is saying is that the agent strongly prefers to see improvements in its solution it dis prefers seeing uh no improvement in its current solution and it is indifferent to um to the unobserved observation it's not exactly how the math would play out because of the softmax function but it's effectively something like that and in any case in this simulation we won't be uh seeing the the unobserved um observation come up at all anyway and then for the second one uh fully uniform preferences uh that simply means that the agent has no particular preference towards um whether it's attending to itself or to its neighbor right so you could experiment with this you could see like what what happens whenever an agent actually prefers to do uh its due diligence and and primarily just explore and come up with its own answers to the to the solution or what if you come up with an agent who more so prefers to steal their or exploit their neighbors's answer in any case here we'll just we'll stay totally um agnostic to uh that and just say that the agent has uniform preferences so those end up looking like this um whenever we print them out uh again these are normalized uh we see how the softmax uh function strongly Amplified uh where where where this uh the the preference for improvement was a five it's now become a 0998 continuing uh probability and so so this relates back to the idea that agents are self- evidencing they will uh based on how particularly expected free energy is

computed uh where it includes C uh within the pragmatic value term um this is what will guide the agent towards uh you know towards observations that it prefers as opposed to others uh making this move Beyond just uh you know an agent who's hyperreactive to what's going on in the present instead of an agent who might uh prefer to uh see certain things and therefore act in a way to kind of realize what it prefers again here uh preference really just meaning it being what you expect um and then again uniform distribution for um for the second uh modality it has no particular preference towards observing itself or uh observing its neighbor as who it's attending to then D fortunately for us we only have one hidden State Factor so this is the shortest description it's similarly to how C was a uh you results in a probability distribution that uh describes the prior over observations the D Matrix does the same thing but for States and we only have one hidden state so we just have one sub array for uh the attention hidden State factor and um you know so again we just uh and I didn't mention this earlier the the np. ons is just generating um a vector uh equivalent to the number of hidden State levels that you have so a vector of two ones and then you're normalizing it using the utils library right so take two ones normalize them you're going to end up with 0.55 the ematrix are your habits uh you don't actually necessarily have to Define these uh whenever you're you're constructing a pmdp agent uh I find it to be very useful if either one you actually want to uh you know uh run some kind of experiment where you think that the agent will actually have more of a a sort of uh a habit of doing one particular action over the other um it's also useful if say you're trying to set up a different kind of simulation that's more focused on uh policies in which case you know it might be the case that you know you only want uh the agent to have a strong prior belief for entire particular policy versus another uh and and use that as a way to to define the agent as having beliefs that you know a particular policy won't at all work versus one that will such as um you know uh dry off and then jump in the pool as a kind of an odd thing to do um or or you know could come up with some better example in the fly but I I think the point comes to cross right so in any case this is just basically your priors over your actions that get Incorporated so what did we do uh we we we related States and observations in the a matrix we related States and actions in the B Matrix uh and then C D and E are just the priors on States uh excuse me observations States and actions respectively um also recall uh from the from the textbook and elsewhere that the agent has a separation from its environment uh called the notion of a marov blanket and so uh for the for the agent UHS like despite certain things occurring during free Min energy minimization to where uh observations and actions do in fact end up influencing one another vicariously uh we have not expressed a probability

distribution that actually directly links observations and actions so it's kind of maintaining the blanket States of uh of your active state sensory States and and you'll you'll see that in the textbook uh reading the the section about markov blankets so it's just maintaining that that kind of Separation then um I will go just a little bit longer than what I expected um so now that we have our a b c d and e um one additional thing that our agent will do is that it will learn uh you know the idea of learning an active inference uh there's plenty of literature on this but it can relate to you know the idea of for the sake of a model what we're doing is that where're we're assigning sort of these hyperparameters or priors to uh to the uh matrices that we just Define themselves right so what our more specifically what our agent will do is it will learn its a matrix the likelihood Matrix that we Define first out of the those five things a B C D and E and uh it it will just update it based on the new observations it receives and the in state that it infers um and it will also answer a question as to okay why did we have set up hidden States and observations and actions in that way why are we doing these like identity mappings between particular things um by learning the a matrix that will allow our agent to learn if it's the case that when it explores it'll be more likely to see an improvement or uh or no improvement right this will allow our agent to by accumulating observations over time in the action perception Loop um it will begin to better update uh its beliefs about which thing it should do and so even though we you know previously we defined e as being uniform over those two actions to where the agent initially had no particular um you know bias towards exploring or exploiting it will actually probably start to bias the agent towards one of those directions because it prefers to see improvements and that can only occur through learning uh without learning and modifying internal uh parameters of your model you're effectively uh you know sort of like a pre-programmed uh uh Thermostat or something right you're nonadaptive you just act in the same way as you always do and so an agent who doesn't learn as the one if we Define an agent like we just did but it never learns then what it will do is more than likely just with a 50/50 chance choose to explore or exploit just keep going uh from there so um the way that we update the a matrix for the PDP agent here um is that we is quite simple and this is it you don't even need all this code you only need the code in the upper right but it's basically this uh small P capital A is the prior that we're creating uh what we're doing is that we're using that util sub Library this de lay like function which basically you construct your a send it through the deerl like function it'll convert it into what is a kind of Exemplar conjugate distribution of a categorical probability distribution uh that being a de lay distribution it it affect whenever you print it out you see that we set scale to one it looks exactly like our a matrix um what that scale

argument does is that it scales the values that's the special thing about the Matrix distribution is that it's something like an accumulation of pseudo counts that will then weigh uh particular um comp parts of the the competition of free energy minimization uh for example the agent might start to see more instances of improvement uh and so it might start to upweight uh the the likelihood of uh Improvement uh given the association that it saw it with you know and so the these don't actually have to sum to one they're just kind of like waiting our any Matrix and they're very relatable to the notion of uh Precision right like if a if an agent hardly has any previous experience uh with anything and it's and it's it's it's U uh prior over a is highly scaled down meaning it it has very few pseudo counts of any prior experience uh it's the agent is going to be less certain in its ability to make connections between uh exactly what the a matrix tries to model which is the relationship between observations and States so um so these values can themselves you know be set differently and and you can set the Precision differently and then they can impact the the agent uh differently now we have all of that this is it for constructing the agent uh it's basically just an agent class and so you just feed in the arguments they're actually many many arguments uh that can go into the agent Constructor um it might seem like a lot there actually uh I think literally a couple dozen more uh but that it very much mirrors the construction of the mat lab SPM package for anyone familiar with that um there's a whole script you know that that will run through and and try and construct your your markup decision- making process your agent and it'll check to make sure that all these uh you know boxes are correct like is the a normalized is you know how did you set Gamma or some other thing and so it's you know in spirit this is very similar to that um what's nice is that the the PDP package um it has different inference algorithms um not many but uh as opposed to choosing sort of the standard one uh I've chosen MMP uh which is marginal message passing there's a nice paper on it that uh can be found in the speaker notes of the tutorial slides I make a brief reference to it here part all 2019 and um so marginal message passing basically being a kind of U you know in a certain way it's a kind of compromise between uh uh overcoming intractability and and attaining achieving something like computational efficiency and approximation that you get with variational message passing but at the same time um it it it it overcomes some of the issues of not um um having certain Precision terms involved that uh you know that leads to issues with VAR variational message passing that it it it can be uh it can lead to overconfident posteriors in the computation and meanwhile belief propagation is much more holistic but um more computationally expensive sometimes tractable depending on the situation so MMP is a nice way of striking balance uh trying to M find that balance

between complexity and and accuracy as we are always trying to do right um so you you'll notice uh that we plugged in everything that we defined so far A B C D E and then the prior over a shows MMP just a little bit more explanation of what some of these other arguments are um but uh given the time and also simply the fact that I mean um we would have to go into a lot more to look the other options available I would say this suffices um you know if you want to play around with any of these um certainly go for it and especially recommend looking at inference Horizon in the policy length uh arguments um this agent is very simple in the sense that it makes no future planning steps for for constructing policies that go beyond just should I explore now or should I exploit now um it it can make inferences one additional time step into the future though so it does in a certain way allow it to have this kind of forward looking n where it'll not just try to anticipate what will happen next but it'll try to anticipate what will happen next next so you know the next time step and the one following that um and uh yeah I also chose stochastic action selection you could choose deterministic instead there's a note on how that works um basically you know if uh if you're posteriors over policies or that uh you know U explore has a value of one you'll choose that with probability .1 right so there's still stochasticity involved um but you're just going to be more likely to choose the option with the highest probability um deterministic means you would automatically go for the the action that has the highest probability assigned in the posterior so those things in mind uh this is the general process uh that will be carried out in um an active inference Loop for our agent it'll infer States uh which you know once we've constructed our agent uh the agent and this is common for other reinforcement learning paradigms as well um you you have an agent and then that agent will have a series of attributes that you can access at any given time um this our agents have an infer States method which can then take in um the observation and then after that uh whenever it comes to learning uh the the PIP install version of imdp has update a b and d matrices uh available um there are uh I I to briefly hearken back to this there have been many updates to the PDP Library over the past couple years it's very exciting but uh I didn't want to give anything too definitive because I think many things are still in progress but right now they're they're acting developing means doing model fitting uh which are already there uh it's just I can see that they're they're kind of cleaning some things up they've also you know there there's a there will be means of using the Jacks back in to speed up a lot of the processes which is very exciting and also they've implemented a sophisticated inference uh which is another way of doing policy inference uh using a kind kind of optimizing the tree search um but I uh in addition to those updates uh they'll also be updating the abilities such

that the agent could learn its preferences or its habits uh or other aspect the aspects these are just the the PIP install versions uh currently available methods then agent will infer its policies and then it will sample its action um so inate inference again we're using marginal message passing um interesting thing about pmdp is that um the only time that F variational free energy really comes up as far as the code goes is that um f gets uh scored across policies rather than being the more General free energy of the overall system per se but in certain ways from looking at the raw code that carries that out uh it appears that you know there are other aspects involved um but for the sake of the tutorial we'll just say what interesting is that uh free energy is the same for both actions at any given point in time and I think that in part is due to the fact that the complexity term uh in uh V defining variational free energy for each um action or policy um it's dependent on this uh the posteriors Over States versus uh your your prior which starts to look like your be Matrix regarding uh State Transitions and the agent has fully uh predictable fully uh you know identity mapping one to one mapping in its B transition matrices and I think that's what actually brings this complexity term uh down to uh virtually zero right uh it's like the the posteriors are basically matching what it it knows what state it'll transition to and if it doesn't ever look learn its B Matrix and never has any opportunity otherwise to see something different from what it already believes then it's as if the agent fully believes like okay you know complexity is gone all it's left with is accuracy and so it's accuracy actually the difference between observations and States that's what's going to lead to the fluctuations in the variational free energy but this is all the code you need a really important part is the way that this OBS Ops uh variable or object is defined again it's very important to keep track of your indices um at any point in time you could you could Define an OBS object like this which is a list the first element of OBS corresponds to the first observation modality which for us was outcome and then the the value itself refers to the index of the particular observation in question so what we're saying here is the first value representing the observation modality outcome modality um that first value zero is index to Improvement and you can see that same logic uh plays out for the second element in the Ops list um you know this is for the attention observation modality and zero representing self as you know what it's indexed to and so it's as if the agent just observed that it was attending to it itself and it saw an improvement we simply run that through uh the agent. in first States method uh and plug into Ops and then the agent will update uh it'll find Qs meaning it's it's posteriors over hidden States and uh those will actually get stored in the agent as an attribute as well but um but the the method Returns the object and so I'm you know we're keeping it

here Q_s and then you can look um we we didn't have to explicitly say anything about Computing F it's just this INF first States um uh run then led to uh an internal computation of f in the agent that we can now access print so this starts getting into um let speed through this a little bit but because I already touched on various things but this is another breakdown you know it's very useful to look at the different breakdowns of uh variational free energy and expected free energy and so on um but I'm GNA move through this slide and come because we'll get back to policy inference shortly uh this uh this is to show that Q_s the posteriors over hidden States uh is rather complex whenever you use the marginal message passing um uh algorithm uh but it's also very interesting in that the the way it gets broken down itself being a rather complex threedimensional matrix it's that the First Dimension refers to policy um if if we if we kind of open it up as it were um and the second one where uh is the time step and the third is the hidden State Factor so what this is saying is that um for the first entry uh which relates to explore and then the time step if you make you know the first entry of that means now and then in state Factor we only have one of them so the first entry is the only entry the attention in state factor that effectively means if you print Q_s uh 0 comma 0 comma 0o uh it'll print out this first value representing this is what the agent believes to be the hidden State given the policy um you know given the Explorer policy and so we can actually extract that out whenever I have these slightly out of line um we can extract that out and then um Thea excuse me um this line here agent. Q_s uh we can we can uh extract it out via Q_s action sampled ID Z all that means is uh we're grabbing the the agent's posterior beliefs given the actual hint state that it chose um that there's a little bit of complexity and it get it's it's much much clearer in the Google collab script um that I've included so I advise having a look at that um but this is the code that allows the agent to uh update its a matrix uh the only reason why uh we need a couple extra lines of code is because of the complex structure of the Q_s that um comes out um of a marginal message passing agent and then we'll look at um um policy inference you saw that there are more slides than what I really need to explain it but um this is the final step of the action perception Loop so for policy inference your posterior beliefs over your policies uh in this case are um your negative expected free energy times gamma which uh have a better slide to represent that here we are um gamma which is your action Precision so just as I mentioned like um we can we can put these uh hyper priors or parameters on our a matrix to learn the a matrix and and and that scale argument kind of kind of um down Tunes or uptunes the Precision on the a matrix similarly here um we can we can downtune or uptune uh the the Precision over G uh expected free energy and since expected free energy is what is

directly involved in realizing one's preferences I mentioned earlier with C matrix it's kind of like whenever we look at this equation is that we're seeing uh a sort of balancing act between expected free energy related to sort of deliberate decision making to realized goals uh you know these are very contextual you know uh kind of beliefs and and and uh ways of computing free energy and then you have variational free energy which uh you know is sort of like the the free energy of the present um and it doesn't necessarily take into account anything regarding uh preferences uh you know the C Matrix and what you say you want to see uh it's more about just kind of optimizing your model in the moment and then uh habits which we already touched on earlier um and then those uh we take the we actually take the natural log of those so that they kind of fit similarly on the same scale with expected free energy and variational free energy so it's kind of like you have these three different you know you have your habits built up from the past you have your variational free energy of the present and then you have your expected free energy of the future all sort of contending with one another and then being run through this soft Max function which as we saw earlier uh when we used it for the C Matrix um that converts it into something that sums to one and then axes our um uh categorical probabilities distribution and the number of entries will be equivalent to the number of actions you have available and so that will look like um in the case of G alone uh it'll it'll look like this you'll have your uh free energy for uh the explore action and your free energy for the exploit action and recall that uh that we want to minimize this quantity so it's actually the the quantity with a smaller number that's uh better so to speak um and then these values will get plugged into the policy inference equation up top in addition to what we already saw earlier the variational free energy values we saw and then the habits which we programmed even earlier than that so we have now all of components for constructing an active inference agent we have all the components needed for doing uh hidden State inference of of having our agent receive observations that it uses to receive uh to to infer hidden States and then to infer policies and to to select an action uh that that's what this slide represents is the ability to uh infer policies um you can print out G and have a look at it uh and then with sample action this is just basically has the agent sample from it's posterities over policies and then this act this extra code here is simply because um you know the whenever you have the agent sample in action it returns it within a list uh and so we need to extract that out of there um and then these last two slides were to represent uh two example simulations that I ran um they're I'm sure they look rather busy as many of these other slides have but there's just so much to kind of look at and get into um with this figure attempts to do is that you can lead it

read it from left to right uh time like over time steps um from the start of a simulation to the end in this case it was 30 time steps long and then you can also read it from top to bottom uh where the agent receives an observation it computes variational free energy um it learns it's a matrix it infers policies and the expected free energy of each action uh it it uh along with that that g then gets included along with f and habits to compute um posterior beliefs about policies and then uh finally the agent chooses an action that gets sent in so it's kind of like from left right we're looking at each time step where a whole uh action perception Loop is happening and then vertically we're looking at an entire uh action perception Loop or you could flip that say perception action Loop given Ag and is getting getting an observation first right and so uh in this simulation I'll just stick with this one uh but uh you know in in this simulation uh the agent um receives an initial observation uh it just forced it to say like self and and Improvement know it seems good um or excuse me it gets initialized with with Improvement and neighbor and then for the first half the simulation we're in what I'll call the neighbor context where in that case uh the simulation is uh you know it's programmed in a loop so the agent will receive um it will always receive uh an improvement if they exploit and they'll always receive a no improvement if they explore so you can see this is a very deterministic uh and very extreme environment where uh you know choosing to explore always leads to a bad outcome for the agent choosing to exploit always leads to a good outcome for the agent and so during the first couple steps call that our agent starts off with um you know these uniform beliefs uh as far as its habits go of whether it should explore or exploit uh and so that's where these red and blue lines and this probability uh subgraph here subplot uh they line up with one another they're 0.5 so effectively the agent is just kind of randomly choosing one of these two options and it just happens it choose to explore both times whenever it chose to explore both times for the next two time steps it saw that there were uh no improvement like bad outcomes um you know given that and uh the the agents free variational free energy of the present uh kind of uh kind of got jerked downward uh in a in a good way it's meaning that it's informative to the agent during those time steps to see that it's accumulating evidence that it's seeing that it might be learning something and in the meantime during the second time step the agent learned its a matrix right so um that also means that there was a lot of updating in its internal model uh that has led to changes in its free energy as well and then over time uh things start to diverge the agent quickly realizes oh if I exploit I receive this green Improvement observation again and again so the agent proceeds to uh exploit and it actually does it all the time as it's doing it it's seeing that there's less and less expected free energy to

just exploit um and then similarly in the final policy inference it it actually reaches um you know a 1.0 probability that the agent should exploit to where the agent is effectively doing this deterministically uh practically given that it keeps inferring that and also this yellow line is especially interesting um and and part of why I wanted to do all this in the first place what this yellow line is doing is that this is me extracting out from it um the probability of the agent seeing an improvement uh given that it attended to its neighbor which links up with exploiting so this is teaching the agen that uh it should keep exploiting um it'll see improvements it'll get What It Wants it'll keep going that way right this agent is adapting to an environment where um you know the that's exactly how this world works right and then halfway through the simulation uh everything is reversed so now uh we went from one environmental extreme to another in the first case the agent saw improvements uh from attending to its neighbor now the agent will only see improvements from attending to itself and it only see no improvements when attending to its neighbor and because there have been so many instances in its past experience of seeing beneficial outcomes when it exploits its neighbor um the agent now actually has to unlearn what it is learned right um and by the end of the simulation still hasn't fully done that um and uh we we can see this reflected in uh the um expected free energy the it starts to rise again for exploit as the agent no longer sees any more improvements even though uh it's continuing to exploit because that's what it expects so this is a toy simulation in that you might not have learning happen after every two times steps that's a very arbitrary thing that I've chosen to do and it's actually a rather rapid learning rate all things considered more often than not whenever setting up a simulation you'll have an agent who might say learn at the end of a trial where you run 10 Trials of Three Time steps each or something along those lines right um it's less common to run trials over 30 time steps however uh in more traditional agent-based modeling uh paradigms because of their Simplicity you often do run uh you know a simulation over many many time steps right and so I want to make sure that this code is set up in a way that you could run this as long as you you know prefer to without without too much complexity because I wanted to show the value to computational social scien as to what they might be able to do um with with active inference agents but now that we're uh significantly Beyond uh you know where where I wanted to wrap up um I'll just take one more minute to say um you know there again there is um you can land at the GitHub uh you know website that I made it's very short to the point and all the links here are uh available for the slides that I was just showing there are many more slides I tried to be rather comprehensive uh given uh you know how much material I'm trying to show and then uh the two collab scripts

with this second one being a further development of the original code but I'm adding more explanatory mark off uh excuse mark off uh markdown uh cells that uh kind of better describe things uh including giving examples of how uh the a matrix works and the B Matrix works and furthermore again I I repeatedly emphasize like being able to get your Matrix construction and structuration down I actually provided code on here's how you create you know an a matrix and you can manually uh Define the values it's a good idea to learn how to manually Define the values of the a matrix or of the me the B Matrix um it looks rather tedious than it is but to be able to do them one by one is a great sort of dactic means of like learning how to construct these including whenever we get into these more complex like ways of um you know representing probability distributions where you have three or more Dimensions but once you're able to do that you might just at some point realize oh there's a loop I could write that could do this for me um but you don't quite know what that Loop looks like until you kind of get a sense of what value should be there right so um anyway so the it's the point is that there are just some additional uh tools and information that can be used here and then all the code is available for say uh printing out like running the example simulations and then looking at the plots at the end feel afraid to you know modify and adapt those um as as pleases you so um that's all I have for today and I would love in future to give another uh talk Andor pre-record something that goes into a lot of the other material and probably gives a more kind of cursory overview of the rest especially for people who would be interested in um multi-agent simulations and um and especially kind of using leveraging packages like Network X which will allow you to kind of connect agents and graphs and Define like sort of like the relationships with one another and it gives clever ways of of giving them actions that are specific to only the agents in their Network or other things like that so um yeah that uh concludes thanks nice great work um why or how would somebody use it other than to get a good handle on the PDP how would you see it being used or adopted for research cool yeah yeah um so I think uh in scenario like this kind of provides a general agent who can like do things that are defined as exploring or exploiting and so um what you could do is use it to you know create the the ink landscape laser Freedman model that I I mentioned earlier uh and then and then tweak things from there or you could use this as a more General template like take out the idea that these are attention and outcome modalities and stuff swap them out for you know whatever you want this can have like the template is here for including as many observation modalities in state factors as you want for example uh just change the names and make sure you define the ab CDs you know uh to your liking and so that that's really what I

wanted to do just create like here here's here's an example agent because we need to know what the final picture should look like but um you know from there just just tear it apart and like you know to to your hearts content and and make it into to what you want to see these agents can be used to you know um the idea of autonomous decision making that if there are any discrete number of things that you'd want to set up some kind of agent to do and you know what kind of data you want it to read um you know and what what should be the impact of each of those discrete actions it has like you have this as a kind of like agent that you can plug into uh that that sort of design diagram right so big answer it's pretty abstract sorry but yeah cool what will be coming up for the next several weeks cool yeah um so with the Symposium I'm looking forward to giving a talk that's a little bit more uh to the script uh and less focused on coding and more on how these kinds of things might be applied in social science research uh including looking at uh you know some other research that has also been done uh just to try and give a a sense that is in fact a sort of subfield forming here and of course we have the the course on social science done with the with the Institute itself last year and um you know different people making Headway in that area so um looking forward to that and uh who who knows what else uh i' very much look forward to maybe presenting some other research where I'm using agents like this and having them interact in kind of interesting ways with let's say LMS um and you know these become a kind of uh fun uh way of trying to imagine autonomous decision making whenever it comes to things like prompting so yeah cool well the video description has links and where things will be versions and it's a great demo so thank you any other comments you um not not currently aside from you know uh thanks everyone who uh joined the textbook group and uh you know over the recent reading uh we're currently talking through uh other ways that we might go about the textbook group going forward but we had a really nice wrapup conversation yesterday so looking forward to Future talks cool okay thank you again Andrew looking forward to the next episodes in the series yeah thanks Daniel thank you peace take care for